Reviewer Pack — Minimal Hodge Tensor Rank and Circuit Monotone Width Inequal…

Entry d53a30986c36 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 17:56:18 UTC

Minimal Hodge Tensor Rank and Circuit Monotone Width Inequality

Entry ID: `d53a30986c36` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-05 17:56:18 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Hodge Theory) **Field B** (complexity object): Boolean Circuit Complexity (Circuit Monotone Width)

Statement:

> For every Boolean circuit C with n inputs and m gates, the minimal Hodge tensor rank of its associated algebraic variety is linearly correlated with its circuit monotone width, such that min_h h(C) = Θ(w_m(C)) for a constant w_0 < w_1 < … < w_m being the increasing sequence of gate counts in C.

Rationale (proposer's reasoning):

> The Hodge tensor rank measures the complexity of algebraic varieties and is connected to the geometry of complex projective spaces. By linking this with circuit monotone width, which quantifies the complexity of Boolean circuits, we may reveal a deep connection between geometric complexity and computational complexity that could lead to new insights into the P vs NP problem.

Taxonomy category: `algebraic_geometry_circuit_monotone_width` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `beca8ed9927d8dd0`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For each circuit, if the ratio between the minimal Hodge tensor rank h(C) and the monotone width w_m(C) is within a factor of n^0.5 over all seeds, support the conjecture; otherwise, falsify it.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Hodge tensor rank" AND "circuit monotone width" - "algebraic geometry" AND "Boolean circuit complexity" - "minimal Hodge tensor rank" AND "0(w_m(C))" AND "circuit monotone width"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_random_boolean_circuit(n, m):
        circuit = []
        for _ in range(m):
            gate_type = random.choice(['AND', 'OR'])
            inputs = random.sample(range(n), 2)
            circuit.append((gate_type, inputs))
        return circuit

    def compute_truth_table(circuit, n):
        truth_table = [[0] * (1 << n) for _ in range(1 << n)]
        for i in range(1 << n):
            inputs = [(i >> j) & 1 for j in range(n)]
            output = 0
            for gate_type, inputs in circuit:
                if gate_type == 'AND':
                    output &= inputs[0] * inputs[1]
                elif gate_type == 'OR':
                    output |= inputs[0] * inputs[1]
            truth_table[i][i] = output
        return truth_table

    def compute_minimal_hodge_tensor_rank(truth_table):
        n = int(math.log2(len(truth_table)))
        if any(row[i] for row in truth_table):
            return 1
        else:
```

```
            return 0

    def compute_monotone_width(circuit, n):
        width = 0
        for gate_type, inputs in circuit:
            if gate_type == 'AND':
                width += 2
            elif gate_type == 'OR':
                width += 2
        return width

    results = []
    for n in [5, 10, 15, 20, 30, 40]:
        for _ in range(5):
            circuit = generate_random_boolean_circuit(n, random.randint(1, n))
            truth_table = compute_truth_table(circuit, n)
            min_h = compute_minimal_hodge_tensor_rank(truth_table)
            w_m = compute_monotone_width(circuit, n)
            results.append((min_h, w_m))

    if not results:
        return {
            "metric_name": "ratio",
            "metric_value": 0,
            "instances_tested": 0,
            "n_max": 40,
            "conjecture_holds": False,
            "counterexample": "empty_results"
        }

    ratio = sum(min_h / w_m for min_h, w_m in results) / len(results)
    return {
        "metric_name": "ratio",
        "metric_value": ratio,
        "instances_tested": len(results),
        "n_max": 40,
        "conjecture_holds": abs(ratio - 1) < 0.5 * n**0.5,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_ratio = sum(result["metric_value"] for result in results) / len(results)
    std_ratio = math.sqrt(sum((result["metric_value"] - mean_ratio)**2 for result in results) / le
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_ratio} std={std_ratio} support_fraction={support_frac
```

```
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_ratio} std={std_ratio} support_fraction={support_frac
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"not_supported\" first_failing_seed={first_faili
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2bb1ea8e.py", line 93, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2bb1ea8e.py", line 63, in run_trial
    min_h = compute_minimal_hodge_tensor_rank(truth_table)
            ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2bb1ea8e.py", line 44, in compute_minim
    if any(row[i] for row in truth_table):
       ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2bb1ea8e.py", line 44, in <genexpr>
    if any(row[i] for row in truth_table):
               ^
NameError: name 'i' is not defined. Did you mean: 'id'?
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed due to an undefined variable 'i', which prevented it from producing any data. As a result, the pre-registered support condition could not be unambiguously met. | next: Debug the test code to fix the NameError and re-run the test to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14103
2	propose	ollama_remote	glm4:latest	0	0	13652
3	preregistration	ollama_remote	glm4:latest	0	0	9373
4	novelty	ollama_remote	glm4:latest	0	0	15502

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
5	novelty	ollama_remote	glm4:latest	0	0	8904
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16370
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13839
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14068
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10743
10	judge	ollama_remote	glm4:latest	0	0	11873

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 128426 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/d53a30986c36.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/d53a30986c36.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/d53a30986c36.tar.gz` (if generated)